

PROJECT DOCUMENTATION

A Project Report Submitted in Partial Fulfilment of the Requirements of the Project Programme

FITBUDDY

AI Fitness Plan Generator Using Gemini Models

Submitted by

Mythili.R (222502791)

Ishwarya.M.G (222502785)

Janani.N (222502786)

Nithish.G (222502769)

Kalai Vani.S (222502787)

Project Title:

FitBuddy – AI Fitness Plan Generator Using Gemini Models

2026

Acknowledgement

We would like to express our sincere gratitude to our project guide and mentor for their valuable guidance, continuous support, and encouragement throughout the course of this project. Their insights greatly contributed to the successful completion of FitBuddy – AI Fitness Plan Generator Using Gemini Models.

We also thank our institution for providing the resources and environment necessary to carry out this project, and Google for making the Gemini Generative AI models accessible through a well-documented API, which forms the core intelligence layer of this application.

Finally, we thank our families and friends for their constant encouragement and support during the development of this project.

Mythili.R
Ishwarya.M.G
Janani.N
Nithish.G
Kalai Vani.S

Abstract

Maintaining a consistent and personalised fitness routine is a challenge for most individuals due to the high cost of professional trainers and the generic, one-size-fits-all nature of fitness content available online. FitBuddy is an AI-powered fitness planning web application designed to address this gap by generating personalised workout plans using Google's Gemini generative AI models. Instead of relying on a fixed library of workout templates, FitBuddy dynamically constructs a workout plan based on the user's name, age, gender, weight, fitness goal, and preferred workout intensity.

The application follows a server-rendered web architecture consisting of a FastAPI backend written in Python, a SQLAlchemy-based database layer for persistent storage, and a Jinja2/HTML/CSS/JavaScript frontend for collecting user input and displaying results. The backend receives the user's fitness profile, constructs a structured prompt, sends it to the Gemini 2.5 Flash model through the Google Generative AI Python SDK, and stores the resulting workout plan in the database for later retrieval.

FitBuddy supports collection of user fitness information, generation of a personalised seven-day workout plan, revision of the generated plan based on user feedback, and

generation of AI-based nutrition tips related to the user's fitness goal. The system demonstrates how a generative AI model can be integrated into a full-stack Python web application to provide fitness assistance without requiring the application itself to implement any fitness-domain machine learning logic.

This documentation presents the problem definition, system analysis, requirement specification, system design, technology stack, frontend and backend implementation, Gemini AI integration, module-wise implementation, testing methodology, and results of the FitBuddy project, followed by its advantages, limitations, and scope for future enhancement.

Keywords

Artificial Intelligence, Generative AI, Google Gemini API, Gemini 2.5 Flash, Fitness Plan Generation, FastAPI, Python, SQLAlchemy, Jinja2, Web Application, Personalised Workout Plan, Nutrition Tips.

Table of Contents

Acknowledgement	2
Abstract	3
Chapter 1 – Introduction	7
Chapter 2 – Literature Survey	9
Chapter 3 – System Analysis	11
Chapter 4 – System Requirements Specification	13
Chapter 5 – System Design	16
Chapter 6 – Technology Stack	19
Chapter 7 – Frontend Design and Implementation	21
Chapter 8 – Backend Design and Implementation	23
Chapter 9 – Gemini AI Integration	25
Chapter 10 – Module-wise Implementation	27
Chapter 11 – Testing	29

Chapter 12 – Results and Discussion 31

Chapter 13 – Advantages and Limitations 32

Chapter 14 – Future Enhancements 33

Chapter 15 – Conclusion 34

References 35

List of Figures

Figure 5.1 – High-level System Architecture of FitBuddy

Figure 5.2 – Level 0 Data Flow Diagram (Context Diagram)

Figure 5.3 – Level 1 Data Flow Diagram

Figure 5.4 – Entity Relationship Design

Figure 7.1 – Frontend Workflow

Figure 7.2 – FitBuddy Home Page (Screenshot)

Figure 7.3 – Fitness Profile Form (Screenshot)

Figure 9.1 – Gemini AI Integration Workflow

Figure 12.1 – Generated 7-Day Workout Plan (Screenshot)

Figure 12.2 – AI Nutrition Tip (Screenshot)

Figure 12.3 – Workout Plan Revision using Feedback (Screenshot)

List of Tables

Table 2.1 – Comparative Summary of Existing Fitness-Planning Approaches

Table 4.1 – Hardware Requirements

Table 4.2 – Software Requirements

Table 5.1 – Database Table Design

Table 6.1 – Technologies at a Glance

Table 8.1 – API Endpoint Summary

Table 10.1 – Team Contribution / Role Table

Table 12.1 – Test Cases and Results

Chapter 1 – Introduction

1.1 Overview

Physical fitness has become an important part of modern lifestyles, yet many individuals find it difficult to design and follow an effective workout routine on their own. The two most common alternatives available today – following generic, pre-written workout plans found online, or hiring a personal trainer – are, respectively, impersonal and expensive. FitBuddy is conceived as a middle path: an AI-powered fitness planning web application that uses Google’s Gemini generative AI models to generate a workout plan tailored to an individual user’s profile, in place of a fixed, one-size-fits-all template.

Instead of manually creating a workout schedule, a FitBuddy user simply provides a small set of fitness-related details – such as their fitness goal and preferred workout intensity – through a simple web form, and the application returns a structured seven-day workout plan generated by the Gemini 2.5 Flash model. The application also allows the plan to be revised based on user feedback and can generate AI-based nutrition tips aligned with the user’s fitness goal.

1.2 Problem Statement

Individuals who wish to begin or maintain a fitness routine frequently face one of the following problems: (a) generic, template-based workout plans found on the internet or in fitness apps do not account for an individual’s specific goal or preferred intensity; (b) hiring a certified personal trainer to design a personalised plan is expensive and not accessible to everyone; and (c) manually researching and assembling a structured, balanced weekly workout routine is time-consuming and requires domain knowledge that many users do not have. There is a need for an accessible, low-cost application that can generate a reasonably personalised workout plan automatically, without

requiring the user to have fitness expertise.

1.3 Objectives of the Project

- To design and develop a web application that collects basic fitness information from the user through a simple form.
- To integrate Google Gemini AI to generate a personalised seven-day workout plan based on the collected fitness information.
- To store user details and the generated workout plan using a relational database accessed through SQLAlchemy.
- To allow the user to submit feedback on the generated plan and receive a revised plan generated by the AI.
- To provide AI-generated nutrition tips relevant to the user's selected fitness goal.
- To build the application using a straightforward, maintainable Python-based technology stack (FastAPI, SQLAlchemy, Pydantic, Jinja2) suitable for an academic project.

1.4 Scope of the Project

The scope of FitBuddy, in its current form, covers collection of a user's fitness profile, AI-generated seven-day workout plan creation using the Gemini API, storage of user and plan data in a database, revision of the generated plan based on textual feedback, and generation of AI-based nutrition tips. The scope does not include user authentication/login, real-time progress tracking, wearable-device integration, video-based exercise-form correction, or medical diagnosis. FitBuddy is a fitness-planning assistance tool and its AI-generated suggestions are not a substitute for professional medical or fitness advice.

1.5 Organization of the Report

This report is organised into fifteen chapters. Chapter 2 surveys related literature and existing fitness-planning approaches. Chapter 3 presents the system analysis, comparing existing and proposed systems along with a feasibility study. Chapter 4 specifies the hardware, software, functional and non-functional requirements. Chapter

5 details the system design, including the architecture, data flow diagrams and database design. Chapter 6 describes the technology stack. Chapters 7 and 8 describe the frontend and backend implementation respectively, with Chapter 8 also covering the REST API design. Chapter 9 focuses on the Gemini AI integration. Chapter 10 describes the module-wise implementation. Chapter 11 covers the testing methodology and test cases. Chapter 12 presents the results and discussion, including screenshot placeholders. Chapter 13 discusses the advantages and limitations of the system, Chapter 14 outlines future enhancements, and Chapter 15 concludes the report, followed by references and an appendix.

Chapter 2 – Literature Survey

2.1 Introduction to the Domain

Digital fitness applications have evolved considerably over the past decade, ranging from simple exercise-logging tools to platforms that attempt some degree of automated personalisation. Before designing FitBuddy, a brief survey of existing approaches to digital fitness planning was carried out to understand their strengths and limitations.

2.2 Survey of Existing Approaches

2.2.1 Template-Based Workout Applications

A large proportion of fitness applications rely on a fixed library of pre-authored workout templates, filtered by a small number of parameters such as goal or experience level. While simple and reliable, the resulting plans are not truly personalised – users with different fitness levels and goals often receive very similar plans.

2.2.2 Human-Coached Platforms

Platforms that connect users with certified human trainers offer genuine personalisation, but at a recurring cost that is not affordable for many users, particularly students and early fitness enthusiasts.

2.2.3 Rule-Based Recommendation Systems

Some tools use rule-based logic (if–then rules built around goal categories and intensity levels) to recommend a plan. These are more flexible than fixed templates but are limited by the number of rules that can practically be authored and maintained.

2.2.4 Generative-AI-Based Fitness Assistants

The most recent generation of fitness tools uses large language models such as Google Gemini to generate plans from a natural-language description of the user’s profile. This approach can produce plans that read naturally and adapt to the specific combination of inputs provided, without requiring the developers to author and maintain an explicit rule set. FitBuddy adopts this generative-AI-based approach, using the Gemini 2.5 Flash model as its plan-generation engine.

2.3 Comparative Summary

Approach	Personalisation	Development Effort	Handles Varied Inputs
Template-based apps	Low	Low	Poor
Human-coached platforms	High	High (ongoing)	Excellent
Rule-based engines	Medium	High (rule authoring)	Fair
Generative-AI (Gemini-based, as in FitBuddy)	High	Medium	Good

Table 2.1: Comparative summary of existing fitness-planning approaches

2.4 Research Gap

The survey highlights a gap between low-effort, low-personalisation template systems and high-personalisation, high-cost human coaching. Generative AI models, when supplied with a structured description of a user’s fitness profile, offer a way to obtain reasonably personalised output without the recurring cost of human involvement or the maintenance burden of a large rule base. This motivates the design of FitBuddy around the Gemini 2.5 Flash model as its core plan-generation mechanism.

Chapter 3 – System Analysis

3.1 Existing System

In the absence of a system such as FitBuddy, a user wishing to start a fitness routine generally relies on generic online articles, fixed workout templates bundled with fitness apps, or a paid personal trainer. These existing approaches suffer from the following drawbacks:

- Workout plans are static and do not adapt to the specific goal or intensity preference of an individual user.
- There is typically no mechanism for a user to give feedback on a generated plan and receive a revised version.
- Nutrition guidance, where available, is usually generic rather than tailored to the user's stated fitness goal.
- Personalised, human-coached alternatives are not affordable for a large number of users.

3.2 Proposed System

FitBuddy addresses these drawbacks by using the Google Gemini generative AI model as a personalisation layer between the user's fitness profile and the final workout plan. The proposed system offers the following improvements over the existing approaches described above:

- On-demand generation of a structured seven-day workout plan based on the user's name, age, gender, weight, fitness goal and workout intensity.
- A feedback-driven revision mechanism, where the user's textual feedback on the generated plan is sent back to the Gemini model to produce a revised plan.
- AI-generated nutrition tips aligned with the user's fitness goal, generated on request.
- Persistent storage of user profiles and generated plans using a SQLAlchemy-backed database, allowing the latest plan for a user to be retrieved at any time.
- A simple, server-rendered web interface (Jinja2 templates with HTML, CSS and JavaScript) that requires no separate frontend build step or installation.

3.3 Feasibility Study

3.3.1 Technical Feasibility

FitBuddy is built using well-established, well-documented, open-source technologies: Python, FastAPI, SQLAlchemy, Pydantic and Jinja2 on the backend, and HTML, CSS and JavaScript on the frontend, together with the officially supported Google Generative AI Python SDK for Gemini access. All of these components are freely available, widely used, and compatible with one another, making the project technically feasible within the time and resource constraints of an academic project.

3.3.2 Economic Feasibility

The primary running cost of FitBuddy is the cost of calling the Gemini API, which is billed on a pay-as-you-go basis and is typically far lower than the cost of hiring a human trainer. All frameworks and libraries used in the project – FastAPI, SQLAlchemy, Pydantic, Jinja2 and the Google Generative AI SDK – are open source and free to use, so development costs are limited to developer time, making the project economically feasible for a student team.

3.3.3 Operational Feasibility

The application presents a simple, single-page form for entering fitness details and displays the generated plan on the same page. Because the interface runs in a standard web browser and does not require user registration or login, it can be used with minimal instruction, supporting strong operational feasibility.

3.4 Requirement Gathering Approach

Requirements for FitBuddy were identified through discussion among the project team on the essential steps of a fitness-planning workflow – collecting user information, generating a plan, allowing revision based on feedback, and providing supplementary nutrition guidance – and were scoped to what could realistically be implemented and demonstrated within an academic project timeline using the chosen Python-based technology stack.

Chapter 4 – System Requirements Specification

4.1 Hardware Requirements

Component	Development Environment	Deployment Environment
Processor	Intel i3 / AMD equivalent or higher	Cloud virtual CPU (shared/ dedicated)
RAM	4 GB minimum (8 GB recommended)	512 MB – 1 GB per service instance
Storage	1 GB free disk space	Storage for SQLite / SQL database file
Network	Stable broadband internet connection (for Gemini API calls)	Public network with HTTPS
Client Device	Any laptop / desktop with a modern web browser	Same

Table 4.1: Hardware Requirements

4.2 Software Requirements

Layer	Technology / Tool	Purpose
Frontend	HTML5, CSS3, JavaScript, Jinja2 Templates	User interface, forms and dynamic HTML rendering
Backend Framework	Python, FastAPI	Application server and REST API endpoints
Data Validation	Pydantic	Request/response schema validation and serialization
Database Layer	SQLAlchemy, SQLite / SQL database	Persistent storage of users, plans and feedback
AI Engine	Google Generative AI Python SDK, Gemini 2.5 Flash	Workout plan generation, plan revision, nutrition tips
Configuration	python-dotenv	Loading the Gemini API key from environment variables
API Server	Uvicorn	ASGI server used to run the FastAPI application
Version Control	Git and GitHub	Source code management
IDE	Visual Studio Code (or equivalent)	Application development
Operating System	Windows / macOS / Linux	Development platform

Table 4.2: Software Requirements

4.3 Functional Requirements

1. The system shall provide a web form for the user to enter fitness details such as name, age, gender, weight, fitness goal and workout intensity.
2. The system shall create or update a user record with the submitted fitness details (POST /users).
3. The system shall send the user's fitness information to the Gemini AI model and generate a personalised seven-day workout plan (POST /generate_plan).
4. The system shall store the generated workout plan in the database, associated with the user.
5. The system shall allow retrieval of a user's most recently generated workout plan (GET /users/{user_id}/latest_plan).
6. The system shall allow the user to submit feedback on the generated plan and receive a revised plan generated by the AI (POST /update_plan).
7. The system shall be able to generate an AI-based nutrition tip related to the user's fitness goal (GET /nutrition_tip).
8. The system shall display the generated workout plan and nutrition tips on the web interface.

4.4 Non-Functional Requirements

4.4.1 Performance

The system shall return an AI-generated workout plan within an acceptable response time under normal network and API load, displaying a loading indicator on the frontend while the Gemini API call is in progress.

4.4.2 Usability

The user interface shall be simple and require only a single form submission for a first-time user to receive a generated workout plan.

4.4.3 Security

The Gemini API key shall be stored as an environment variable (loaded via python-

dotenv) and shall never be committed to source control or exposed to the client browser; the .env file shall be listed in .gitignore.

4.4.4 Reliability

The system shall handle failures or unexpected responses from the Gemini API gracefully, avoiding an application crash, and shall validate incoming request data using Pydantic schemas before processing.

4.4.5 Maintainability

The backend codebase shall be organised into separate modules for the application entry point (main.py), AI logic (ai_service.py), database configuration (database.py), database models (models.py), request/response schemas (schemas.py) and database operations (crud.py), so that individual concerns can be maintained independently.

Chapter 5 – System Design

5.1 System Architecture

FitBuddy follows a layered client-server architecture. The presentation layer consists of HTML pages rendered through Jinja2 templates, styled with CSS and made interactive using JavaScript, running in the user's browser. The application layer is a FastAPI backend that exposes REST-style API endpoints, validates incoming data using Pydantic, and coordinates calls to the AI service and the database layer. The data layer consists of a SQLAlchemy-backed SQL database (SQLite) that persists user information and generated workout plans. The AI layer is the Google Gemini 2.5 Flash model, accessed through the Google Generative AI Python SDK from within ai_service.py.

[Figure: FitBuddy System Architecture]

Figure 5.1: High-level system architecture of FitBuddy

The frontend never communicates with the Gemini API directly; every AI request passes through the FastAPI backend, which keeps the Gemini API key confidential on the server side and allows the backend to validate and structure the prompt before it is

sent to the model.

5.2 Data Flow Diagram

5.2.1 Level 0 DFD (Context Diagram)

[Figure: Level 0 Data Flow Diagram]

Figure 5.2: Level 0 DFD – User interacts with the FitBuddy system, submitting fitness details and receiving workout plans and nutrition tips.

5.2.2 Level 1 DFD

At Level 1, the FitBuddy system is decomposed into the following major processes: (1) Create/Update User, which stores the submitted fitness profile in the database; (2) Generate Workout Plan, which sends the user profile to the Gemini AI service and stores the resulting plan; (3) Retrieve Latest Plan, which reads the most recent plan for a user from the database; (4) Update Plan from Feedback, which sends the existing plan and user feedback to the Gemini AI service to produce a revised plan; and (5) Generate Nutrition Tip, which requests goal-based nutrition guidance from the Gemini AI service.

[Figure: Level 1 Data Flow Diagram]

Figure 5.3: Level 1 DFD showing the major processes of FitBuddy

5.3 Database Design

The database layer is implemented using SQLAlchemy, defined in models.py, and stores user information, generated workout plans and user feedback. The exact fields and relationships correspond to the models defined in the project's models.py file.

Entity	Purpose	Managed By
User	Stores the fitness profile submitted by a user (e.g., name, age, gender, weight, goal, intensity).	models.py / crud.py
Workout Plan	Stores the AI-generated seven-day workout plan associated with a user.	models.py / crud.py
Feedback	Stores user feedback submitted for plan revision.	models.py / crud.py

Table 5.1: Database entity overview (see Chapter 8 for full field-level detail as implemented in models.py)

[Figure: Database/ER Design]

Figure 5.4: Entity relationship overview of the FitBuddy database

5.4 Module Design

The backend is organised into the following cooperating modules, each implemented as a separate Python file:

- main.py – FastAPI application setup, routing, template and static file serving, and API endpoint definitions.
- ai_service.py – Gemini API configuration, prompt construction, workout plan generation, plan revision and nutrition tip generation.
- database.py – database engine and session management.
- models.py – SQLAlchemy model definitions for users, workout plans and feedback.
- schemas.py – Pydantic schemas for request validation and response serialization.
- crud.py – Create, Read, Update and Delete operations against the database.

5.5 UI Flow

At a high level, the frontend flow consists of the following steps for a user: (1) the user opens the FitBuddy home page, served from templates/index.html; (2) the user fills in the fitness profile form (name, age, gender, weight, goal, workout intensity); (3) the form is submitted via JavaScript (static/js/script.js) to the backend, which creates/updates the user and generates a workout plan; (4) the generated seven-day plan is displayed on the page; (5) the user may optionally submit feedback to request a revised plan, or request an AI-generated nutrition tip.

Chapter 6 – Technology Stack

6.1 Overview

FitBuddy is built using a Python-centric full-stack architecture: a FastAPI backend, a SQLAlchemy-based data layer, a Jinja2/HTML/CSS/JavaScript frontend, and the

Google Gemini 2.5 Flash model as the AI intelligence layer. Using Python across the backend and templating layer keeps the codebase compact and consistent for a small academic project team.

6.2 Frontend Technologies

- HTML5 – used for the page structure and the fitness-profile form.
- CSS3 – used for styling, layout and the overall visual appearance of the interface.
- JavaScript – used for client-side interactivity, form submission and communication with the backend API (static/js/script.js).
- Jinja2 Templates – used for dynamic HTML rendering through FastAPI (templates/index.html).

The frontend is a traditional server-rendered interface enhanced with JavaScript for interaction; it is not built using a component framework such as React.

6.3 Backend Technologies

- Python – the primary programming language used for the backend.
- FastAPI – the web framework used to define API routes, serve templates and static files, and handle requests and responses.
- SQLAlchemy – the Object Relational Mapper (ORM) used to define database models and interact with the SQL database.
- SQLite / SQL database – used for persistent storage of users, workout plans and feedback.
- Pydantic – used to define and validate request and response schemas.
- Jinja2 – used by FastAPI to render the HTML templates.
- python-dotenv – used to load the Gemini API key and other configuration from a .env file.
- Uvicorn – the ASGI server used to run the FastAPI application.

6.4 AI Engine – Google Gemini

The AI engine used by FitBuddy is Google's Gemini generative AI model, accessed

through the official Google Generative AI Python SDK. The specific model used in the project is Gemini 2.5 Flash, configured in `ai_service.py` using `genai.GenerativeModel("gemini-2.5-flash")`. The Gemini 2.5 Flash model was chosen for its ability to generate structured, natural-language fitness content quickly in response to a well-formed prompt built from the user's profile.

6.5 Version Control

Git and GitHub are used for source code management and collaboration among the team members during the development of FitBuddy.

6.6 Technologies at a Glance

Category	Technology
Programming Language	Python
Frontend	HTML5, CSS3, JavaScript, Jinja2
Backend Framework	FastAPI
Database Layer	SQLAlchemy
Database	SQLite / SQL database
Data Validation	Pydantic
AI Engine	Google Gemini
AI Model	Gemini 2.5 Flash
Environment Management	python-dotenv
API Server	Uvicorn
Version Control	Git and GitHub

Table 6.1: Technologies at a glance

Chapter 7 – Frontend Design and Implementation

7.1 Overview

The FitBuddy frontend is a lightweight, server-rendered web interface implemented using HTML5, CSS3 and JavaScript, delivered through Jinja2 templates rendered by the FastAPI backend. The frontend does not use a JavaScript framework such as React; instead, a single HTML page is rendered on the server and enhanced with JavaScript

for dynamic behaviour.

7.2 Responsibilities of the Frontend

- Collecting the user's fitness information through an HTML form.
- Sending the submitted fitness information to the backend using JavaScript (fetch/AJAX-style calls) from static/js/script.js.
- Displaying the AI-generated seven-day workout plan returned by the backend.
- Sending user feedback to the backend for workout plan revision.
- Displaying AI-generated nutrition tips.
- Providing a simple, responsive fitness-planning interface.

7.3 templates/index.html

templates/index.html is the main web page of the application. It is rendered by FastAPI through Jinja2 and provides the fitness-profile form used to collect the user's name, age, gender, weight, fitness goal and workout intensity, along with the sections in which the generated workout plan and nutrition tips are displayed.

7.4 static/css/style.css

style.css is responsible for the visual presentation of the application, including page styling, layout of the form and results sections, colour scheme, buttons, and the overall appearance of the fitness dashboard displayed on the page.

7.5 static/js/script.js

script.js handles all client-side interactivity. Its responsibilities include capturing form submissions, sending the collected fitness information to the appropriate FastAPI endpoint, receiving the generated workout plan and nutrition tips from the backend, and updating the page dynamically to display the results without a full page reload.

[Figure: Frontend Workflow]

Figure 7.1: Frontend workflow – form submission, API call, and dynamic display of results

[Insert Screenshot: FitBuddy Home Page]

[Insert Screenshot: Fitness Profile Form]

Chapter 8 – Backend Design and Implementation

8.1 Overview

The FitBuddy backend is implemented in Python using the FastAPI framework. It is responsible for receiving fitness information from the frontend, validating it, coordinating with the Gemini AI service to generate and revise workout plans, persisting data to the database, and returning results to the frontend.

8.2 main.py

main.py is the main FastAPI application file. Its responsibilities include creating the FastAPI application instance, connecting the application to the database, serving the Jinja2 HTML templates and static files, and defining the API endpoints described below.

8.3 API Design

FitBuddy exposes the following REST-style API endpoints, all defined in main.py:

Method	Endpoint	Purpose
POST	/users	Creates or updates a user’s fitness profile.
POST	/generate_plan	Generates a personalised workout plan using Gemini AI.
GET	/users/{user_id}/latest_plan	Retrieves the user’s most recently generated workout plan.
POST	/update_plan	Updates/revises a workout plan based on user feedback.
GET	/nutrition_tip	Generates an AI-based nutrition tip.

Table 8.1: API endpoint summary

8.4 database.py

database.py manages the database connection for the application. It is responsible for creating the SQLAlchemy database engine, creating database sessions, and providing database access to the FastAPI request-handling functions through dependency

injection.

8.5 models.py

models.py contains the SQLAlchemy model definitions that describe the structure used to store user information, workout plans and feedback in the database.

8.6 schemas.py

schemas.py contains the Pydantic schemas used for input validation, data serialization, and defining the structure of API request and response bodies.

8.7 crud.py

crud.py contains the Create, Read, Update and Delete (CRUD) database operations used by the application, including creating and reading users, updating users, creating and reading workout plans, updating workout plans, and saving user feedback.

8.8 requirements.txt

requirements.txt lists the Python dependencies required to run the project, including FastAPI, Uvicorn, SQLAlchemy, Pydantic, Jinja2, the Google Generative AI SDK and python-dotenv. Dependencies are installed using the command:

```
pip install -r requirements.txt
```

8.9 test_models.py

test_models.py is used for testing the database models and the application's data structures.

Chapter 9 – Gemini AI Integration

9.1 Overview

FitBuddy uses Google Gemini as its main artificial intelligence technology. The AI integration is implemented entirely within ai_service.py, which uses the Google Generative AI Python library to communicate with the Gemini model.

9.2 AI Model

The project uses the Gemini 2.5 Flash model, configured in `ai_service.py` as follows:

```
genai.GenerativeModel("gemini-2.5-flash")
```

9.3 AI Features

9.3.1 Personalised Workout Generation

The AI service receives information such as the user's name, age, gender, weight, fitness goal and workout intensity, and uses this information to build a prompt that is sent to the Gemini model. The model generates a structured seven-day workout plan based on this prompt.

9.3.2 Workout Plan Revision

The user can provide feedback about the generated workout plan through the frontend. This feedback, along with the existing plan, is sent back to the Gemini model via the `/update_plan` endpoint, which creates a revised plan reflecting the feedback.

9.3.3 Nutrition Tips

The application can request AI-generated nutrition advice from the Gemini model, tailored to the user's stated fitness goal, through the `/nutrition_tip` endpoint.

9.4 AI Workflow

[Figure: Gemini AI Integration Workflow]

Figure 9.1: AI workflow — User Fitness Details → FastAPI Backend → AI Service → Google Gemini AI → Personalised Workout Plan → Database → Frontend Display

9.5 What Makes the Project AI-Based

FitBuddy is not simply a fitness form backed by a fixed set of templates; the personalisation layer is the core AI component of the application. The user's fitness information is converted into a natural-language prompt, which the Gemini model analyses in order to generate a customised workout plan. For example, a user profile with a weight-loss goal and moderate intensity produces a different seven-day plan

than a profile with a different goal or intensity, because the specific values are embedded in the prompt sent to Gemini. The AI can further revise this plan when the user submits feedback, rather than the application relying on a static set of rules.

9.6 API Key Configuration

The Google Gemini API key is stored as an environment variable and loaded using python-dotenv, rather than being hard-coded in the application:

```
GOOGLE_API_KEY=your_api_key_here
```

The real API key is never committed to the project's GitHub repository; the environment file (.env) is listed in the project's .gitignore file to prevent accidental disclosure.

Chapter 10 – Module-wise Implementation

10.1 Project Structure

The FitBuddy project follows the directory structure shown below:

```
FitBuddy-AI-Fitness-Plan-Generator-Using-Gemini-Models/  
  main.py  
  ai_service.py  
  crud.py  
  database.py  
  models.py  
  schemas.py  
  requirements.txt  
  test_models.py  
  templates/  
    index.html  
  static/  
    css/  
      style.css  
    js/  
      script.js
```

10.2 User Profile Module

Implemented across main.py (the POST /users endpoint), schemas.py (input

validation), models.py (the User model) and crud.py (create/read/update operations), this module is responsible for collecting and persisting the user's fitness profile.

10.3 Workout Plan Generation Module

Implemented across main.py (the POST /generate_plan endpoint), ai_service.py (prompt construction and Gemini API call) and crud.py (saving the generated plan), this module produces and stores the personalised seven-day workout plan.

10.4 Plan Retrieval Module

Implemented via the GET /users/{user_id}/latest_plan endpoint in main.py together with a corresponding read operation in crud.py, this module retrieves the most recently generated plan for a given user.

10.5 Plan Revision Module

Implemented via the POST /update_plan endpoint in main.py, this module accepts user feedback, passes it to ai_service.py to generate a revised plan through the Gemini model, and updates the stored plan through crud.py.

10.6 Nutrition Tip Module

Implemented via the GET /nutrition_tip endpoint in main.py and the corresponding function in ai_service.py, this module generates an AI-based nutrition tip aligned with the user's fitness goal.

10.7 Team Contribution

The FitBuddy project was developed collaboratively by the following team members. Individual module-wise contributions are marked as [To be added].

Name	Register Number	Contribution
Mythili.R	222502791	[To be added]
Ishwarya.M.G	222502785	[To be added]
Janani.N	222502786	[To be added]
Nithish.G	222502769	[To be added]

Kalai Vani.S	222502787	[To be added]
--------------	-----------	---------------

Table 10.1: Team contribution / role table

Chapter 11 – Testing

11.1 Testing Methodology

Testing of FitBuddy was carried out at the module level, focusing on the correctness of the database models and application data structures (via test_models.py) and on manual, functional verification of each API endpoint – user creation/update, workout plan generation, plan retrieval, plan revision and nutrition tip generation – to confirm that each endpoint behaves as expected for valid input and handles invalid or missing input without crashing the application.

11.2 Levels of Testing

- Unit Testing – verifying the SQLAlchemy database models and data structures defined in models.py, exercised through test_models.py.
- Integration Testing – verifying that the FastAPI endpoints in main.py correctly coordinate the AI service (ai_service.py) and the database layer (crud.py, database.py).
- Functional / Manual Testing – exercising each API endpoint with representative fitness profiles to confirm that a valid seven-day workout plan, plan revision, or nutrition tip is produced.

11.3 Test Cases

Test Case	Input	Expected Result	Status
Create/update user	Valid fitness profile (name, age, gender, weight, goal, intensity)	User record is created or updated in the database	[To be added]
Generate workout plan	Valid user profile submitted to / generate_plan	A structured 7-day workout plan is returned and stored	[To be added]
Retrieve latest plan	Valid user_id submitted to /users/{user_id}/latest_plan	The most recently generated plan for that user is returned	[To be added]

Revise workout plan	Existing plan and feedback text submitted to / update_plan	A revised workout plan reflecting the feedback is returned and stored	[To be added]
Generate nutrition tip	Valid fitness goal submitted to / nutrition_tip	An AI-generated nutrition tip relevant to the goal is returned	[To be added]
Invalid/missing input	Incomplete fitness profile submitted to / users or / generate_plan	A validation error is returned by Pydantic without crashing the server	[To be added]

Table 12.1: Test cases (results to be filled in after formal test execution)

Chapter 12 – Results and Discussion

12.1 Results Overview

The FitBuddy application successfully demonstrates the intended workflow: a user's fitness profile, submitted through the web form, is used to construct a prompt for the Gemini 2.5 Flash model, which returns a structured seven-day workout plan that is displayed on the frontend and persisted in the database. The plan-revision and nutrition-tip features similarly demonstrate that the Gemini model can be invoked for different fitness-related purposes from within the same backend service (ai_service.py).

12.2 Discussion

The results confirm that a relatively small, Python-based full-stack application – combining FastAPI, SQLAlchemy and a single external Gemini API integration – is sufficient to deliver a working, personalised fitness-planning experience without the need for the application itself to implement fitness-domain logic or maintain a rule base. Because the personalisation is driven entirely by the prompt sent to Gemini, the quality and structure of the generated plan is closely tied to the design of the prompt built in ai_service.py.

12.3 Screenshots / Output

The following screenshots illustrate the working application. [Actual screenshots to be

inserted after final testing.]

[Insert Screenshot: Generated 7-Day Workout Plan]

[Insert Screenshot: AI Nutrition Tip]

[Insert Screenshot: Workout Plan Revision using Feedback]

Chapter 13 – Advantages and Limitations

13.1 Advantages

- **AI-Based Personalisation** – workout plans are generated according to each user's individual fitness information rather than a single fixed plan being reused for every user.
- **AI Feedback Loop** – users can submit feedback and receive a revised workout plan generated by the AI, rather than being limited to a single static plan.
- **Goal-Based Planning** – the generated plan takes the user's stated fitness goal and preferred workout intensity into account.
- **AI Nutrition Support** – the system can provide nutrition tips related to the user's selected fitness goal in addition to the workout plan.
- **Full-Stack AI Application** – the project integrates a web frontend, a Python backend, a database, and a generative AI service into a single cohesive application.
- **Lightweight Technology Stack** – the use of FastAPI, SQLAlchemy and server-rendered Jinja2 templates keeps the application simple to run and maintain compared with a separate frontend build pipeline.

13.2 Limitations

- The application does not currently implement user authentication or login; fitness profiles are managed without a secure account system.
- The application does not include long-term progress tracking, body-metric history, or trend charts.
- The quality of the generated workout plan is dependent on the Gemini model's response and the design of the prompt; the AI-generated content is not verified by a certified fitness professional.

- The application requires an active internet connection and a valid Gemini API key to generate or revise workout plans and nutrition tips.
- FitBuddy provides general fitness and nutrition suggestions and is not a substitute for professional medical or fitness advice.

Chapter 14 – Future Enhancements

The following enhancements are identified as possible directions for extending FitBuddy beyond its current scope:

- Adding user authentication and secure login so that fitness profiles and plans are tied to individual user accounts.
- Introducing a progress-tracking module that logs completed workouts and body-weight readings over time.
- Extending the AI integration to consider historical progress data when generating or revising a plan.
- Deploying the application to a cloud hosting platform for wider accessibility.
- Adding automated unit and integration test coverage beyond the current `test_models.py`.
- Enhancing the frontend styling and responsiveness for a broader range of device sizes.

Chapter 15 – Conclusion

FitBuddy demonstrates how a generative AI model – specifically, Google’s Gemini 2.5 Flash – can be integrated into a full-stack Python web application to provide personalised fitness assistance. By combining a FastAPI backend, a SQLAlchemy-based database layer, a Jinja2/HTML/CSS/JavaScript frontend, and the Google Generative AI Python SDK, the project delivers a working system that collects a user’s fitness profile, generates a personalised seven-day workout plan, allows the plan to be revised based on feedback, and provides AI-generated nutrition tips.

The project brings together web development, Python backend programming, database

management, REST API design, and applied generative AI in a single, coherent academic project, and provides a foundation that can be extended with authentication, progress tracking and further AI-driven personalisation in future iterations.

References

- [1] FastAPI Documentation. Available at: <https://fastapi.tiangolo.com/>
- [2] Python Software Foundation. Python 3 Documentation. Available at: <https://docs.python.org/3/>
- [3] SQLAlchemy Documentation. Available at: <https://docs.sqlalchemy.org/>
- [4] Pydantic Documentation. Available at: <https://docs.pydantic.dev/>
- [5] Jinja2 Documentation. Available at: <https://jinja.palletsprojects.com/>
- [6] Google AI for Developers. Gemini API Documentation. Available at: <https://ai.google.dev/gemini-api/docs>
- [7] Google Generative AI Python SDK. Available at: <https://github.com/google-gemini/generative-ai-python>
- [8] Uvicorn Documentation. Available at: <https://www.uvicorn.org/>
- [9] MDN Web Docs. HTML, CSS and JavaScript References. Available at: <https://developer.mozilla.org/>
- [10] python-dotenv Documentation. Available at: <https://pypi.org/project/python-dotenv/>